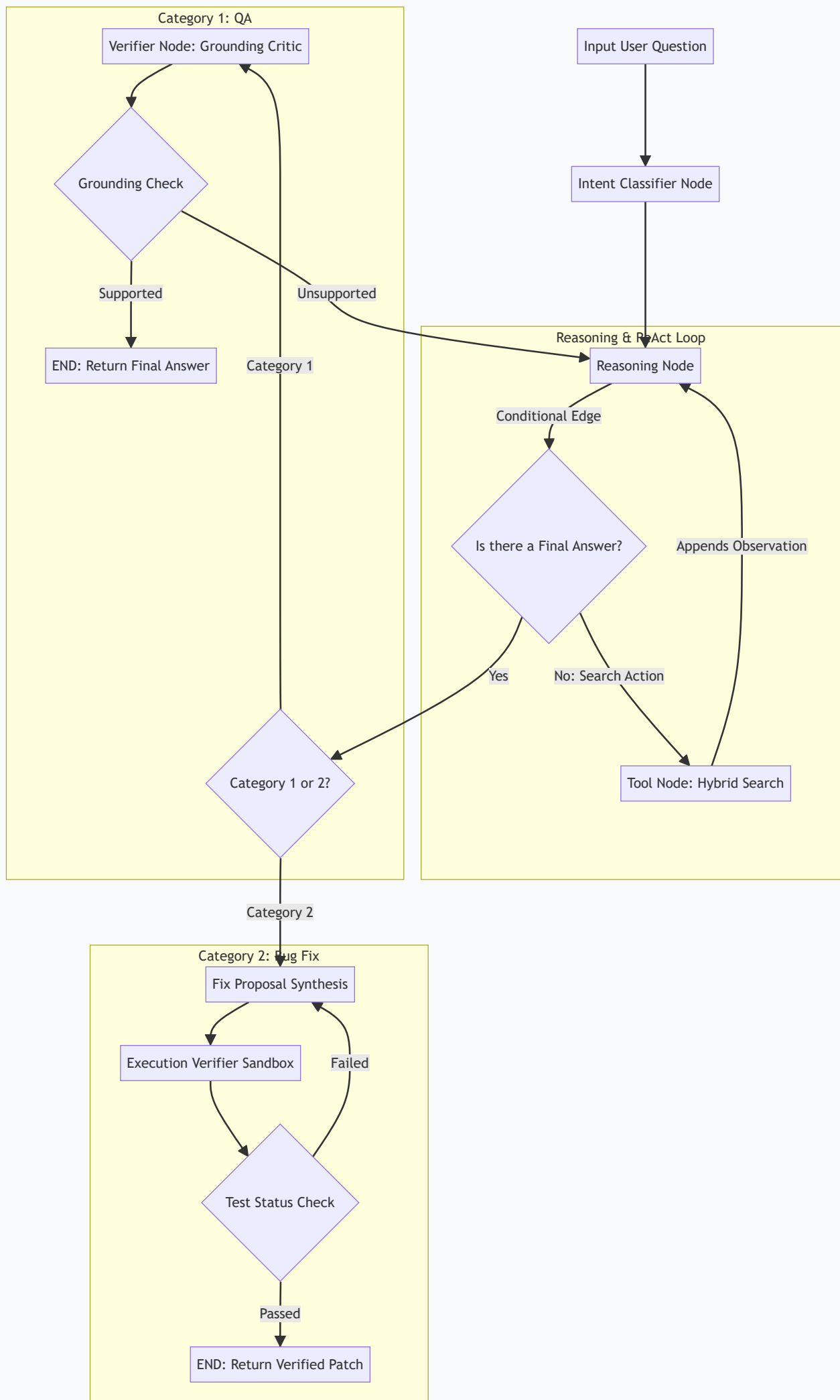


AI Coding Agent Architecture & Journey

Complete Technical Documentation & System Evolution

1. Dual-Mode ReAct Pipeline Workflow

The system intelligently routes the user's request, executes multi-turn semantic searches, and verifies output through independent Critic models or Python execution sandboxes. This prevents unverified hallucinated code from reaching the user.



2. The Journey: Architectural Decisions & Motives

Every decision in this RAG and Agent pipeline was made to combat real-world codebase limitations discovered during empirical testing.

2.1 AST-Based Structural Chunking

Decision: Use Tree-sitter to parse the Python AST and chunk code at the method-level.

Motive: Massive files dilute embeddings. A 1,900-line class compresses 22,000 tokens into a single vector, causing blind spots. Character windows break Python syntax. Method-level chunking preserves structural context while drastically shrinking average chunk size to 422 tokens.

2.2 Hybrid Search with RRF Fusion

Decision: Combine dense embeddings (ChromaDB) with sparse match (BM25).

Motive: LLMs struggle to map abstract queries to specific programmatic identifiers. Dense search captures meaning, while BM25 handles exact variable lookup. Reciprocal Rank Fusion ensures the best of both worlds bubble to the Top-20 chunks.

3. The 2-Day Debugging Saga: Syntax & File Loader Conundrum

During the development of the Sandbox, we encountered a critical failure loop that stalled progress for 2 days. The system generated valid fixes, but the Execution Verifier repeatedly rejected them. We identified the root causes:

The File Loader Problem (Whitespace & Newline Mismatches)

The Issue: To inject a fix, the agent uses verbatim substring match (``string.replace()``). However, LLMs generate text with varying whitespace (`` `` vs `` ``). The strict string replacement failed silently.

The Solution: We implemented a normalization step in the file loader. Before attempting a substring match, the agent strips all trailing whitespace line-by-line (``line.rstrip()``) for both the source file and the LLM's snippet.

The Syntax Problem (Markdown Contamination)

The Issue: Even when the patch was successfully applied, the sandbox immediately crashed with a `SyntaxError` during the `ast.parse()` pre-check. The LLM was wrapping its replacement code in Markdown codeblocks, causing the AST to fail.

The Solution: We implemented a robust regex-based `clean_code_snippet()` function that automatically detects and strips markdown tags from the LLM's output before parsing.

4. Multi-Model Allocation Stack

We transitioned from monolithic models to a specialized architecture to maximize speed, prevent infinite loops, and guarantee safety.

Node Name	Configured Model	Rationale & Architectural Motive
Intent Classifier	stepfun-ai/step-3.7-flash	Instant Routing: Binary QA vs Bug Fix decision requires low latency. Step-3.7-flash completes this in ~0.3s, eliminating queueing bottlenecks.
Reasoning Node	minimaxai/minimax-m3	High-Intelligence Meta-Cognition: Small 8B models suffer from infinite repetitive search loops. Minimax M3 possesses the high-capacity planning required to navigate complex repositories in 4-6 turns.
Verifier Node	thinkingmachines/inkling	Strict Guardrails: Requires absolute determinism. Inkling acts as an exceptional fact-checking auditor, ensuring zero hallucinations slip into the final answer.
Execution Verifier	minimaxai/minimax-m3	Robust Test Generation: Needs to synthesize edge-case heavy pytest scripts and interpret raw stderr tracebacks. Minimax M3 adapts test cases effectively.